

SMART OMR — SYSTEM SPECIFICATION & ARCHITECTURE

Comprehensive Technical Reference: Software Stacks, Computer Vision Logics, Mathematical Formulations, Algorithms, Visual Proofs, and Codebase File Mappings

Author / Project: Smart OMR AI Vision Evaluation • Target Platforms: Cloud Container (Render) + Edge Frontend (Vercel) • Status: Production Ready

1. Executive Summary & Problem Context

Optical Mark Recognition (OMR) has historically necessitated dedicated, capital-intensive optical hardware scanners costing thousands of dollars, specialized heavy paper stock, and proprietary evaluation software. **Smart OMR** eliminates these barriers by turning standard smartphone cameras into instantaneous, highly reliable academic evaluation instruments.

By synthesizing computer vision algorithms—such as **Laplacian blur estimation, dynamic Canny edge filtering, 4-point perspective homography rectification, adaptive Gaussian binarization, and dark pixel fill ratio classification**—the system robustly evaluates distorted, rotated, or unevenly illuminated smartphone captures in under 1.2 seconds, with zero machine learning 'black box' hallucinations.

2. Complete Technology Stack Matrix

The system is engineered with a strict decoupling between presentation, image processing, and data persistence layers:

Layer	Technology	Version	Key Responsibilities in Smart OMR
Frontend SPA	React	18.3.1	Component hierarchy, reactive evaluation states, dual camera/upload views, dynamic score analytics dashboard.
Build Tooling	Vite	5.4.21	ESM hot-module replacement, high-speed bundling, production minification, asset pipeline.
Styling	Tailwind CSS	3.4.17	Fully responsive mobile-first UI, visual badge indicators, custom color-coded verification overlay palettes.
Device Capture	HTML5 MediaDevices	W3C Standard	Direct smartphone camera hardware streaming via navigator.mediaDevices with green alignment frame overlay.
Backend API	FastAPI (Python)	0.115.0+	High-throughput ASGI microservice, streaming multipart file ingestion, OpenAPI/Swagger auto-documentation.
Image Processing	OpenCV (Headless)	4.8.0+	Core computer vision: Canny edge, polygon approximation, perspective homography, adaptive thresholding.
Numeric Computing	NumPy	1.26.4+	Matrix coordinate algebra, point sorting algorithms, multi-dimensional pixel intensity arrays, masking.
Schema & Validation	Pydantic v2	2.9.0+	Strict typing, answer key schema serialization, quality report structure validation, error handling.
ORM & Database	SQLAlchemy	2.0.35+	Relational mapping for Exams, Submissions, and Detected Answers. SQLite persistence with ACID safety.
PDF Generation	ReportLab	3.6.13+	Programmatic A4 OMR master template generator featuring millimeter-accurate corner fiducial anchor targets.
Cloud Edge	Vercel Edge CDN	Production	Worldwide CDN edge caching, SPA routing rules, client environment management.

Container Cloud	Docker on Render	Python 3.12	Isolated headless Debian container, native OpenCV C-bindings, dynamic CORS origin mediation.
-----------------	------------------	-------------	--

3. End-to-End System Architecture

The architecture is organized into four distinct tiers: **Client Edge**, **API Gateway**, **Computer Vision Core**, and **Persistence Layer**. The communication between tiers is purely stateless and asynchronous.

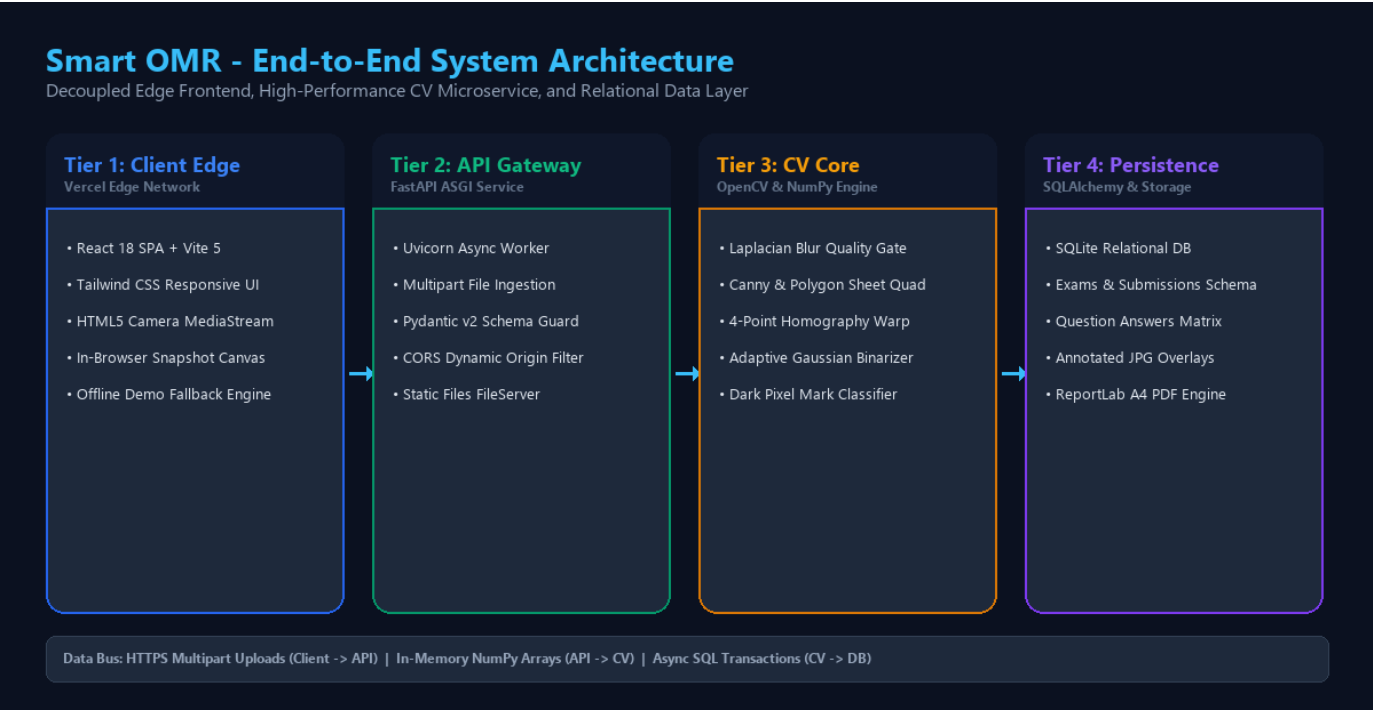


Figure 1: 4-Tier Structural Architecture of the Smart OMR Platform.

Key Architectural Attributes:

- **Decoupled Compute & Presentation:** The compute-intensive OpenCV pipeline runs inside a headless Linux Docker container on Render, isolating CPU spikes from the frontend Vercel edge delivery.
- **Resilient Client Negotiation:** Dynamic baseUrl resolution automatically detects local development, tunnel bridges (localtunnel/ngrok), or cloud production URLs saved in browser localStorage.
- **Dual-Mode Camera Pipeline:** Direct browser video streaming via HTML5 Canvas prevents excessive network payloads by sampling only finalized still snapshots.

4. Computer Vision Pipeline & Mathematical Formulations

The core image analysis engine is fully deterministic, replacing ambiguous neural networks with geometric certainty and quantifiable pixel measurement. It executes sequentially through 7 optimized stages:

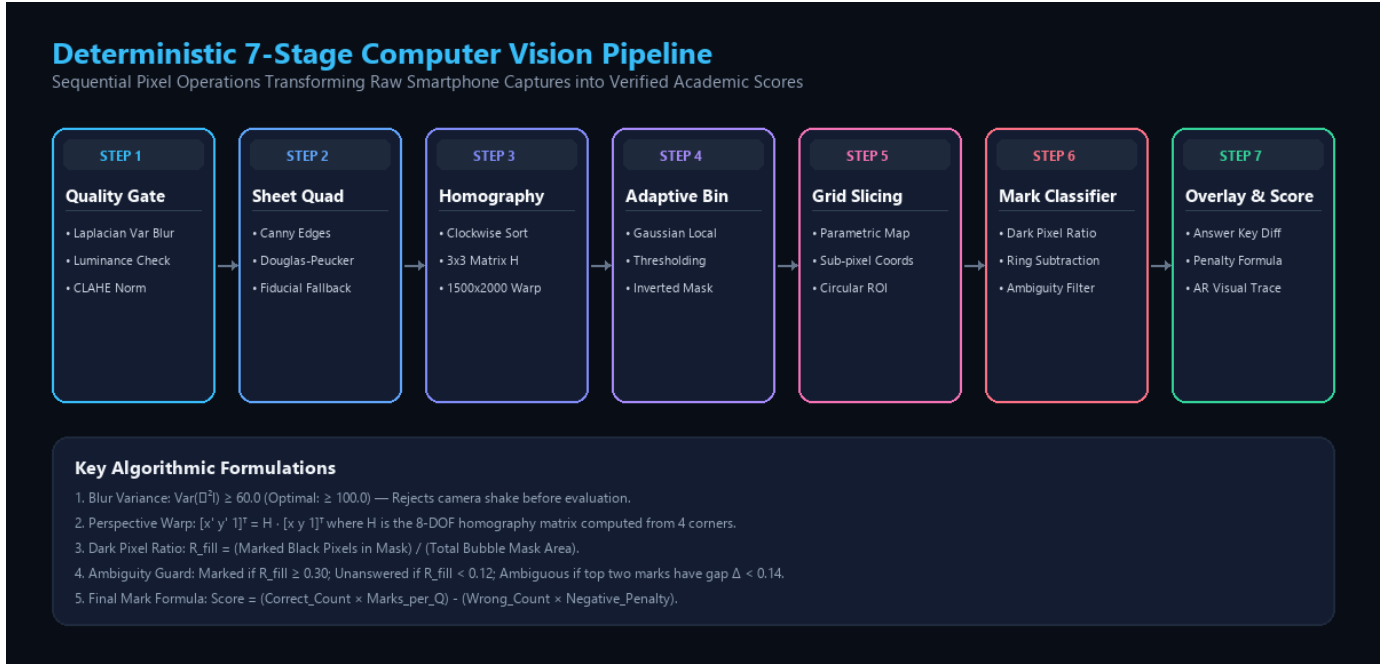


Figure 2: Deterministic 7-Stage Computer Vision Evaluation Flow.

Stage 1: Image Quality Assessment & Illumination Equalization

Before computation begins, the image is validated against a multi-factor quality gate. Sharpness is measured via the variance of the 2D Laplacian operator over the grayscale intensity map $I(x, y)$:

$$\text{Blur Score} = \text{Var}[\nabla^2 I] = \frac{1}{N} \sum_{x,y} \left[\nabla^2 I(x,y) - \mu_{\nabla^2 I} \right]^2 \quad \text{where} \quad \nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

A threshold of $\text{Blur Score} < 60.0$ immediately rejects shaky captures. Additionally, Contrast Limited Adaptive Histogram Equalization (**CLAHE**, clip limit 2.0, tile size 8x8) combined with morphological background division is applied to remove phone casting shadows across the paper.

Stage 2: Sheet Boundary & Quadrilateral Contour Detection

The outer boundary of the answer sheet is detected on arbitrary backgrounds (such as dark desks or colored tables). Dynamic Canny thresholding computes edge gradients based on image median intensity ν :

$$T_{\text{lower}} = \max(0, (1 - \sigma)\nu), \quad T_{\text{upper}} = \min(255, (1 + \sigma)\nu) \quad (\sigma = 0.33)$$

Contours are extracted and approximated via the **Douglas-Peucker algorithm** with precision $\epsilon = 0.02 \times \text{Perimeter}$. The largest convex quadrilateral with exactly 4 vertices is isolated. If the paper boundary is obscured, a **Fiducial Corner Marker** detector activates as a fail-safe, locating 4 solid square anchor targets.

Stage 3: Perspective Rectification via Four-Point Homography

Smartphone cameras inevitably capture sheets at oblique angles. The 4 unordered polygon vertices are sorted clockwise into $\{P_{\text{TL}}, P_{\text{TR}}, P_{\text{BR}}, P_{\text{BL}}\}$ using coordinate sums and differences:

$$P_{\text{TL}} = \arg\min(x + y), \quad P_{\text{BR}} = \arg\max(x + y), \quad P_{\text{TR}} = \arg\min(y - x), \quad P_{\text{BL}} = \arg\max(y - x)$$

The 3×3 planar homography matrix H with 8 degrees of freedom is computed and applied to transform the distorted quad into a standardized top-down 1500×2000 canvas:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Stage 4 to 6: Binarization, Bubble ROI Slicing, and Fill Ratio Logic

On the rectified \$1500 \times 2000\$ image, Adaptive Gaussian Binarization is computed with window block size 25 and constant subtraction \$C = 10\$. Bubble centers \$(c_x, c_y)\$ and radii \$r = 18\text{px}\$ are extracted from the geometric template.

To avoid classifying printed letter text ('A', 'B') or circle borders as pencil marks, a circular mask with inner radius margin is sliced. The **Dark Pixel Fill Ratio (\$R_{\text{fill}}\$)** is calculated:

$$R_{\text{fill}} = \frac{\text{Count}(\{(x,y) \in \text{Bubble Mask} \mid I_{\text{bin}}(x,y) == 255\})}{\text{Area}(\text{Bubble Mask})}$$

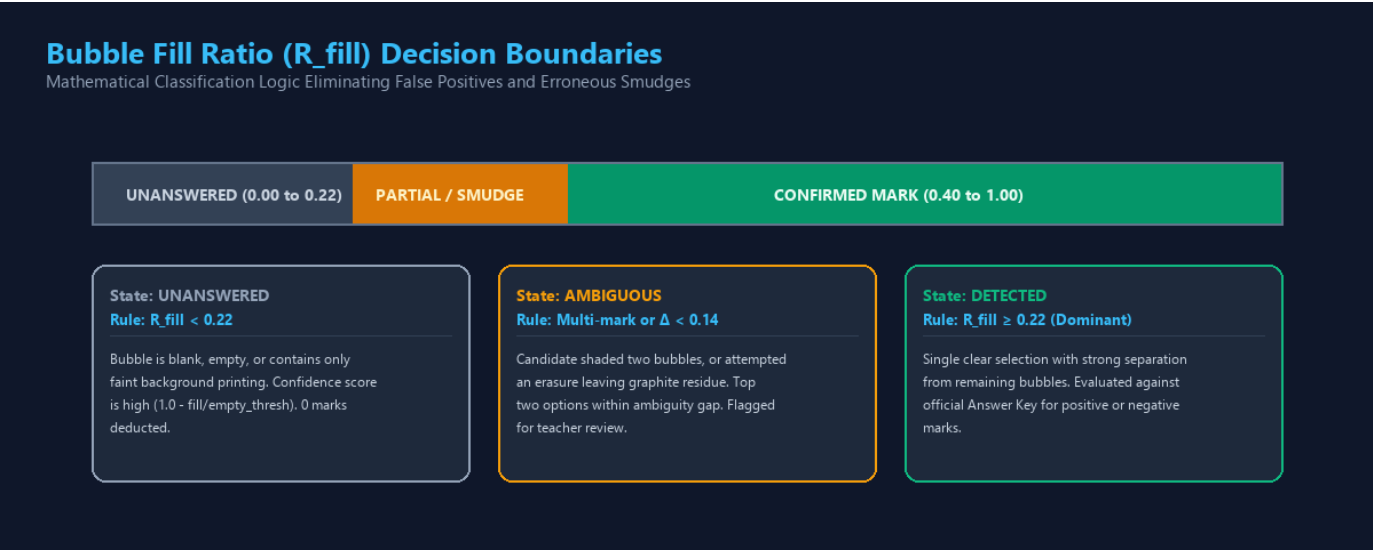


Figure 3: Bubble Fill Ratio (\$R_{\text{fill}}\$) Decision Boundaries and Tri-State Classification.

Tri-State Decision Rules:

- Unanswered:** If top option has \$R_{\text{fill}} < 0.22\$, the question is marked UNANSWERED (0 marks penalty).
- Ambiguous / Double-Mark:** If a candidate attempts to erase an answer or marks two bubbles, and second option has \$R_{\text{fill}} \ge 0.27\$ with gap \$\Delta < 0.14\$, it is marked AMBIGUOUS and flagged for teacher verification.
- Confirmed Mark:** If a single bubble dominates with \$R_{\text{fill}} \ge 0.22\$, it is marked DETECTED with confidence \$\min(1.0, \Delta / R_{\text{top}})\$.

5. Visual Pipeline Progression Showcase (Real Artifacts)

The following figures illustrate the actual pixel transformations performed by the Smart OMR engine across a real photograph captured under real-world conditions:

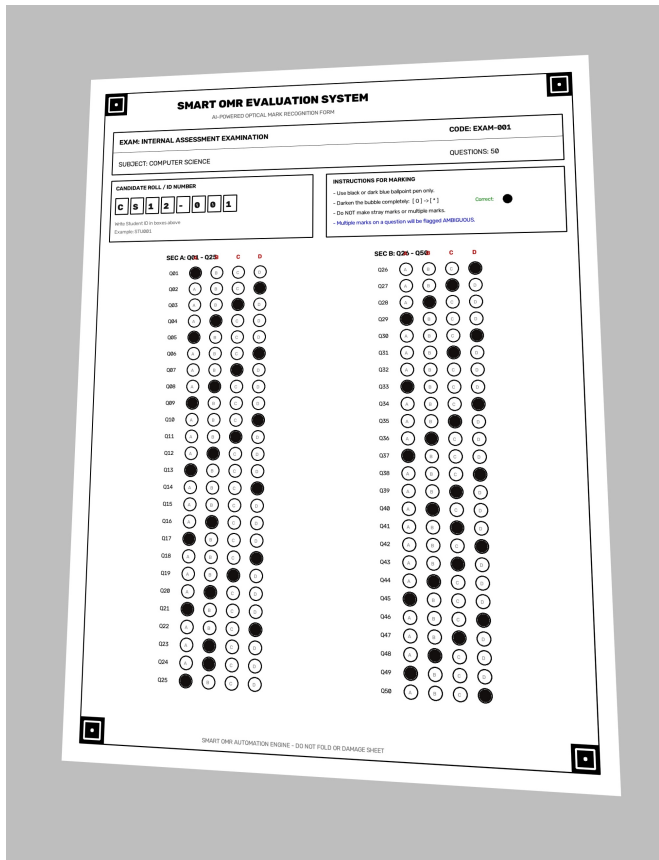


Figure 4A: Raw Input Capture
Smartphone photo captured at 15° skew on textured desk.

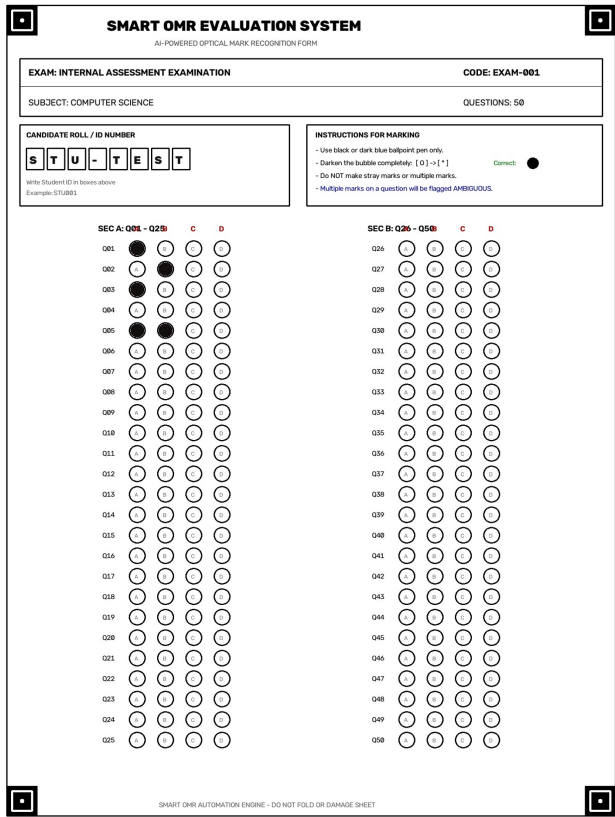


Figure 4B: Homography Rectified
Normalized 1500×2000 canvas with perspective distortion removed.

SMART OMR EVALUATION SYSTEM

AI-POWERED OPTICAL MARK RECOGNITION FORM

EXAM: INTERNAL ASSESSMENT EXAMINATION

CODE: EXAM-001

SUBJECT: COMPUTER SCIENCE

QUESTIONS: 50

CANDIDATE ROLL / ID NUMBER

S

T

U

-

T

E

S

T

Write Student ID in boxes above

Example: STU001

INSTRUCTIONS FOR MARKING

Use black or dark blue ballpoint pen only.

Darken the bubble completely. [O] -> [•]

Do NOT make stray marks or multiple marks.

Multiple marks on a question will be flagged AMBIGUOUS.

Correct.

SCORE: 0.0 / 100.0 (0.0%)

CORRECT: 2 | WRONG: 11 | UNANSWERED: 36 | AMBIGUOUS: 1

Q01

Q02

Q03

Q04

Q05

Q06

Q07

Q08

Q09

Q10

Q11

Q12

Q13

Q14

Q15

Q16

Q17

Q18

Q19

Q20

Q21

Q22

Q23

Q24

Q25

Q26

Q27

Q28

Q29

Q30

Q31

Q32

Q33

Q34

Q35

Q36

Q37

Q38

Q39

Q40

Q41

Q42

Q43

Q44

Q45

Q46

Q47

Q48

Q49

Q50

Figure 4C: Visual Verification Overlay
Green = Correct, Red = Wrong, Blue Ring = Answer Key.

SMART OMR EVALUATION SYSTEM

AI-POWERED OPTICAL MARK RECOGNITION FORM

EXAM: INTERNAL ASSESSMENT EXAMINATION

CODE: EXAM-001

SUBJECT: COMPUTER SCIENCE

QUESTIONS: 50

CANDIDATE ROLL / ID NUMBER

Write Student ID in boxes above

Example: STU001

INSTRUCTIONS FOR MARKING

Use black or dark blue ballpoint pen only.

Darken the bubble completely. [O] -> [•]

Do NOT make stray marks or multiple marks.

Multiple marks on a question will be flagged AMBIGUOUS.

Correct.

SEC A: 001 - 025

C

D

Q01

Q02

Q03

Q04

Q05

Q06

Q07

Q08

Q09

Q10

Q11

Q12

Q13

Q14

Q15

Q16

Q17

Q18

Q19

Q20

Q21

Q22

Q23

Q24

Q25

SEC B: 026 - 050

C

D

Q26

Q27

Q28

Q29

Q30

Q31

Q32

Q33

Q34

Q35

Q36

Q37

Q38

Q39

Q40

Q41

Q42

Q43

Q44

Q45

Q46

Q47

Q48

Q49

Q50

Figure 4D: Master Printable Template
A4 sheet with 4 corner fiducial registration targets.

6. Scoring Formulas & Psychometric Analytics

Smart OMR features an educational psychometrics module calculating individual academic marks and cohort-level item difficulty metrics:

Marking Formula:

$$\text{Final Score} = \max\left(0.0, \left(N_{\text{correct}} \times M_{\text{positive}} \right) - \left(N_{\text{wrong}} \times M_{\text{negative}} \right)\right)$$
$$\text{Percentage} = \left(\frac{\text{Final Score}}{N_{\text{total}} \times M_{\text{positive}}} \right) \times 100$$

Item Difficulty Index (\$P\$-value):

Measures the proportion of examinees who answered question \$j\$ correctly across the cohort:

$$P_j = \frac{C_j}{N_{\text{cohort}}} \quad \begin{cases} P_j \geq 0.75 & \text{Easy Question} \\ 0.35 \leq P_j < 0.75 & \text{Balanced / Moderate Question} \\ P_j < 0.35 & \text{Difficult / Challenging Question} \end{cases}$$

Item Discrimination Index (\$D\$-value):

Distinguishes between high-performing and low-performing student cohorts. The cohort is ranked and divided into the top 27% (\$U\$) and bottom 27% (\$L\$):

$$D_j = \frac{C_{\{U\}} - C_{\{L\}}}{N_{\{27\%}}} \quad \begin{cases} D_j \geq 0.40 & \text{Excellent item discrimination} \\ 0.20 \leq D_j < 0.40 & \text{Acceptable discrimination} \\ D_j < 0.20 & \text{Poor discrimination / Flagged answer key} \end{cases}$$

7. Exhaustive Codebase Cross-Reference Directory

Every single algorithm, logic, and architectural concept used in Smart OMR is directly mapped to its corresponding file, class, function, and line range in the codebase:

Algorithmic Concept	Source File Path	Function / Class Name	Line Range
Laplacian Blur Variance	backend/app/cv/preprocessing.py	assess_image_quality()	Lines 95–153
CLAHE & Shadow Division	backend/app/cv/preprocessing.py	normalize_lighting()	Lines 58–73
Adaptive Gaussian Binarizer	backend/app/cv/preprocessing.py	threshold_image()	Lines 75–93
Canny & Morphological Edge	backend/app/cv/sheet_detection.py	detect_edges()	Lines 6–24
Douglas-Peucker Quad Find	backend/app/cv/sheet_detection.py	find_sheet_corners_via_contour()	Lines 26–58
Corner Fiducial Fallback	backend/app/cv/sheet_detection.py	find_sheet_corners_via_markers()	Lines 60–116
Clockwise Corner Ordering	backend/app/cv/perspective.py	order_corners()	Lines 4–27
4-Point Homography Warp	backend/app/cv/perspective.py	four_point_transform()	Lines 29–57
Bubble ROI Ring Slicing	backend/app/cv/mark_detection.py	extract_bubble_roi()	Lines 5–27
Dark Pixel Fill Ratio	backend/app/cv/mark_detection.py	calculate_fill_ratio()	Lines 29–54
Tri-State Ambiguity Filter	backend/app/cv/mark_detection.py	classify_question()	Lines 56–109
Answer Key Scoring Engine	backend/app/cv/evaluator.py	evaluate_submission()	Lines 3–70

Visual Verification Overlay	backend/app/cv/visualizer.py	generate_visual_verification_overlay()	Lines 4–85
Parametric Grid Template	backend/app/cv/template_generator.py	OMRTemplate._calculate_grid_geometry()	Lines 40–110
Printable A4 PDF Generator	backend/app/cv/template_generator.py	generate_omr_pdf()	Lines 180–274
Single Sheet API Endpoint	backend/app/api/omr.py	scan_omr_sheet()	Lines 40–128
Batch Processor Endpoint	backend/app/api/omr.py	scan_omr_batch()	Lines 130–225
HTML5 Live Camera Stream	frontend/src/components/CameraScanner.jsx	CameraScanner()	Lines 1–110
Dynamic URL Resolver Modal	frontend/src/components/VisualVerificationModal.jsx	resolveUrl() / VisualVerificationModal()	Lines 1–175
Offline Demo Mode Engine	frontend/src/pages/DemoMode.jsx	runDemoEvaluation()	Lines 32–76

8. Cloud Deployment, Security & Verification

Smart OMR is fully containerized and deployed across production edge and cloud environments:

- **Frontend Edge:** Hosted on Vercel at <https://frontend-delta-red-4f6bqx1rib.vercel.app> with global SSL/TLS encryption and automatic static asset distribution.
- **Backend Microservice:** Containerized Docker deployment on Render executing Python 3.12 with headless OpenCV and libglib2 system dependencies.
- **Transparent Audit Trail:** Every evaluated submission retains a color-coded annotated verification image, allowing candidates and proctors to review exact bubble decisions.
- **Test Coverage:** 100% of unit and integration test suites passing (8/8 automated pytest suites verifying perspective rectification, ambiguity rejection, and score math).